



Programação web

Criando nossas primeiras APIs

Prof. Diego Cardoso Borda Castro



Estrutura do projeto

- sudo apt update
- sudo apt install nodejs
- node -v
- Gerenciador de pacote
 - sudo apt install npm





Nossa primeira API

- npm init
 - Package.json
- Pasta src
 - Index.js
 - node index.js



```
const http = require("http");
const url = require("url");

function listar_dados(req, res) {
  const parsedUrl = url.parse(req.url, true);

  if (parsedUrl.pathname === "/listar" && req.method === "GET") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end("Nossa primeira API");
  }
}

const server = http.createServer(listar_dados);
server.listen(3000);
```

SEMANA 1

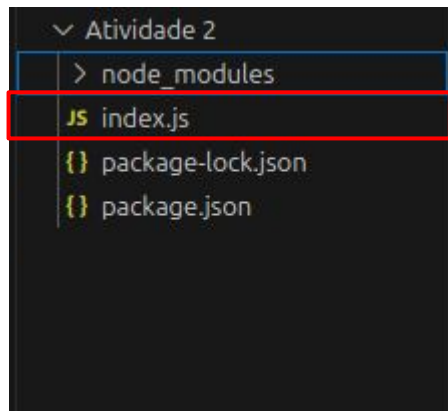
Atividade 1

- js index.js
- {} package-lock.json
- {} package.json



Express

- npm i express



```
Atividade 2 > js index.js > listar
1  const express = require("express");
2
3  const app = express();
4
5  app.get("/listar", listar);
6
7  function listar(req, res){
8      console.log("Nossa primeira API 2");
9      return res.json("Nossa primeira API 2");
10 }
11
12 app.listen(3000);
13
```



Express

<code>app.get("rota", função)</code>	Listar
<code>app.get("rota/:id", função)</code>	Visualizar
<code>app.post("rota", função)</code>	Adiconar
<code>app.put("rota/:id", função)</code>	Atualizar
<code>app.patch("rota/:id", função)</code>	Atualizar
<code>app.delete("rota/:id", função)</code>	Deletar



Adicionando um item

```
1  app.post("/livro", (req, res) => {  
2    const { nome, autor } = req.body;  
3  
4    livros.push({ nome, autor, id: livros.length });  
5  
6    res.status(201).send();  
7  });
```

Método

Rota

Captura do request

Retorno



Pegando um item



```
1 app.get("/livro", (_, res) => {  
2   res.status(200).json(livros);  
3 });  
4  
5 app.get("/livro/:id", (req, res) => {  
6   res.status(200).json(livros[req.params.id]);  
7 });
```

Retorno

Captura do request



Editando um ítem

```
1 app.put("/livro/:id", (req, res) => {  
2   const { nome, autor } = req.body;  
3  
4   livros[req.params.id] = { nome, autor, id: req.params.id };  
5  
6   res.status(200).send();  
7 });  
8
```

Desestruturação



Editando um ítem

- PUT
 - Atualização completa
- PATCH
 - Atualização parcial

```
1  app.patch("/livro/:id", (req, res) => {
2    const { nome } = req.body;
3    const id = req.params.id;
4    const livro = livros[id];
5
6    if (nome)
7      livro.nome = nome;
8
9    res.status(200).send();
10 });
```



Deletando um ítem



```
1  app.delete("/livro/:id", (req, res) => {  
2    livros.splice(req.params.id, 1);  
3  
4    res.status(200).send();  
5  });
```



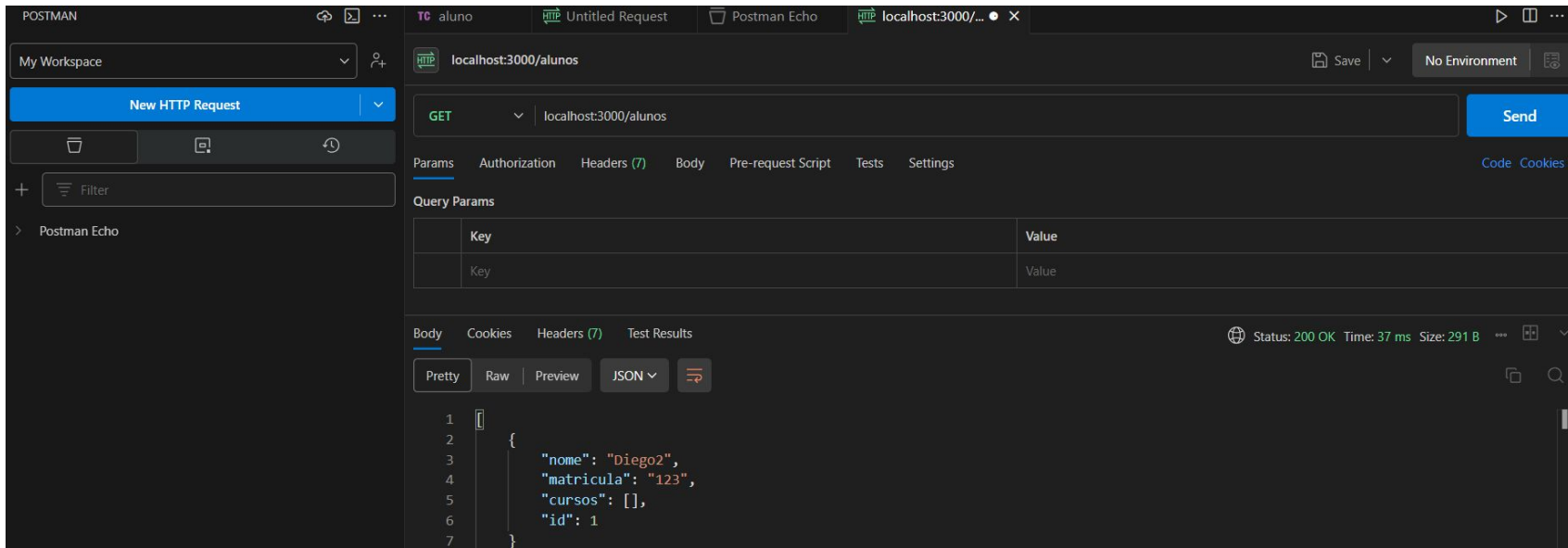
HTTP

- Browser
 - Apenas [GET]
- Postman
- Insomnia
- Thunder client
 - Requisição http
 - Environment

```
1  const express = require("express");
2
3  const app = express();
4
5  app.get("/alunos", (request, response) => {
6      return response.json([
7          { "nome": "aluno 1" },
8          { "nome": "aluno 2" },
9          { "nome": "aluno 3" },
10     ])
11 })
12
13 app.post("/alunos", (request, response) => {
14     return response.json(
15         { "nome": "aluno 4" }
16     )
17 })
18
19 app.put("/alunos/:id", (request, response) => {
20     return response.json(
21         { "nome": "aluno 4" }
22     )
23 })
24
25 app.patch("/alunos/:id", (request, response) => {
26     return response.json(
27         { "nome": "aluno 4" }
28     )
29 })
30
31 app.delete("/alunos/:id", (request, response) => {
32     return response.json(
33         { "message": "Aluno 4 deletado" }
34     )
35 })
36
37 app.listen(3000);
```



HTTP



Postman



HTTP

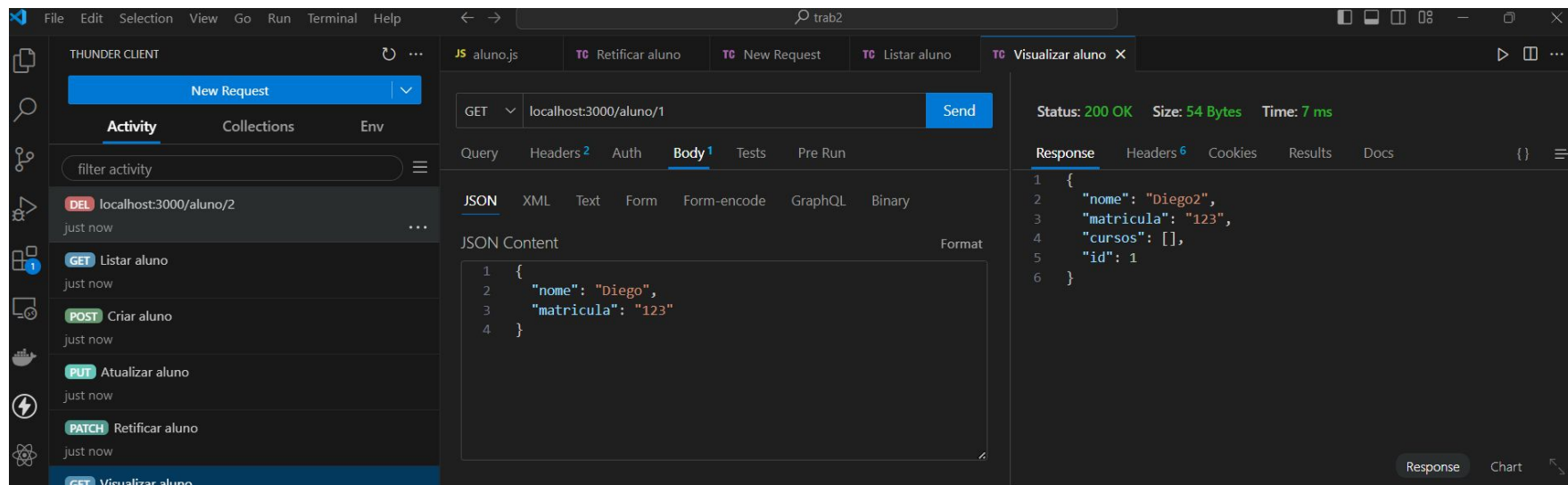
The screenshot displays the Insomnia REST client interface. On the left sidebar, the 'Base Environment' is set to 'cefet-aula'. Below it, there are options to 'Add Cookies' and 'Add Certificates'. A 'Filter' input field is present. A list of endpoints is shown under the 'alunos' folder, including 'delete' (DEL), 'atualizar' (PATCH), 'atualizar' (PUT), 'listar' (GET), and 'criar' (POST). The main panel shows a 'DELETE' request to the URL 'http://localhost:3000/aluno/1'. The 'URL PREVIEW' section shows the full URL. Below that, there are sections for 'QUERY PARAMETERS' and 'PATH PARAMETERS'. The 'PATH PARAMETERS' section includes a note: 'Path parameters are url path segments that start with a colon ':' e.g. ':id''. The right sidebar shows the response status '200 OK', response time '4.58 ms', and response size '30 B'. The 'Preview' tab is active, showing a JSON response:

```
{  "message": "Aluno 4 deletado"}
```





HTTP



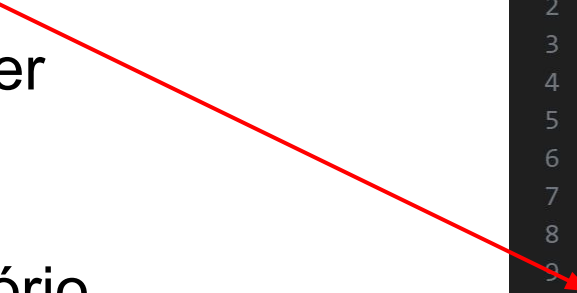
Thunder client



Organização de código

- Camadas de abstração

- Index
- Router
- Controller
- DTO
- Service
- Repositório
- Entidade



```
1  import express from "express";
2
3  import { CriarAluno, ListarAlunos }
4  from "../alunos/alunoController";
5
6  const app = express();
7  app.use(express.json());
8
9  app.post('/CriarAluno', CriarAluno)
10 app.get('/ListarAluno', ListarAlunos)
11
12 app.listen(3000, () =>
13   console.log(`Servidor na porta: ${3000}`)
14 )
```



Organização de código

- Camadas de abstração

- Index

- Router
 - Controller
 - DTO
 - Service
 - Repositório
 - Entidade

```
1  const express = require('express') // Importa o express
2  const rotas = require('./routes/index.js');
3
4  const app = express() // Instancia o express
5  const port = 3000 // Porta onde vai rodar o express
6
7  app.use(express.json())
8
9  app.use('/', rotas); // Utiliza as rotas
10
11 app.listen(port, () => // Executa o express na porta definida
12     console.log(`Example app listening on port ${port}!`)
13 )
```




Organização de código

- Camadas de abstração

- Index
- Router
- Controller
- DTO
- Service
- Repositório
- Entidade

Importa as rotas das entidades

```
1  const express = require('express'); // Import do express
2  const rotas = express.Router(); // Importa do criador de rotas
3  const alunos = require('./aluno');
4  const cursos = require('./curso');
5
6  rotas.use('/', alunos); // definição padrão de rotas
7  rotas.use('/', cursos);
8
9  module.exports = rotas; // Export do módulo para importar no index
10
```



Organização de código

- Camadas de abstração

- Index
- Router
- Controller
- DTO
- Service
- Repositório
- Entidade

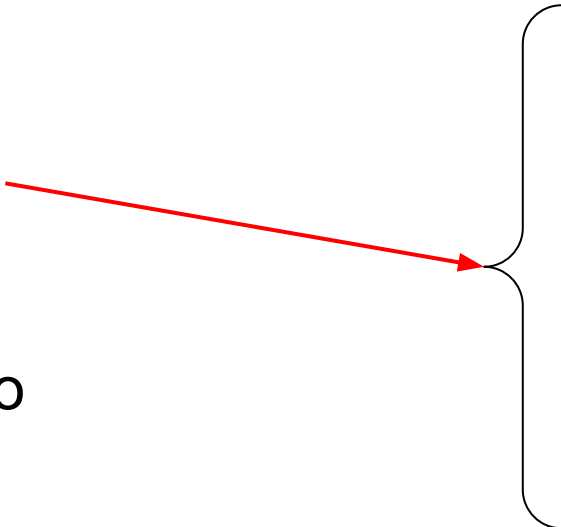


```
1  const express = require('express'); // Import do express
2  const rotas = express.Router(); // Importa do criador de rotas
3
4  const controller = require('../controllers/curso.js'); // Import do controller
5
6  rotas.get('/cursos', controller.listar); // Definição das rotas
7  rotas.get('/curso/:id', controller.visualizar);
8  rotas.post('/curso', controller.criar);
9  rotas.put('/curso/:id', controller.atualizar);
10 rotas.patch('/curso/:id', controller.retificar);
11 rotas.delete('/curso/:id', controller.deletar);
12
13 module.exports = rotas; // Export do módulo para importar no index
14
```

Rotas



Organização de código

- Camadas de abstração
 - Index
 - Router
 - **Controller**
 - DTO
 - Service
 - Repositório
 - Entidade
- 
- Receber a Requisição
 - Enviar resposta
 - Comunicação com a camada de regras

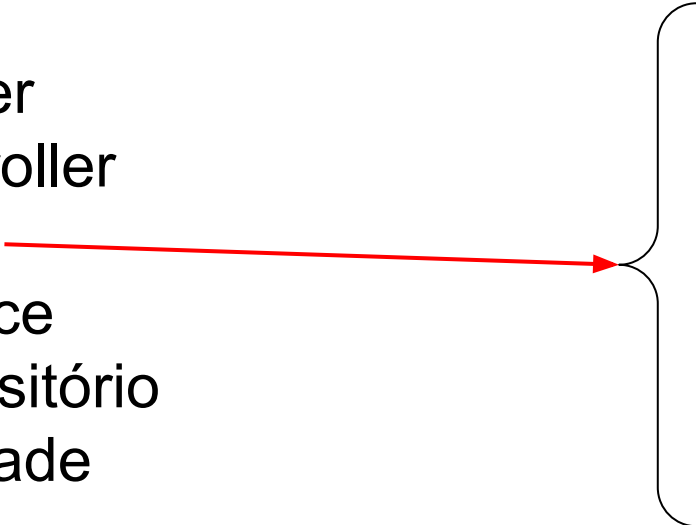


Organização de código

```
1  import { Request, Response } from "express"
2  import alunoService from "../alunoModel"
3
4  export async function ListarAlunos(request: Request, response: Response) {
5      const alunos = await alunoService.listar();
6      response.status(201).json(alunos);
7  }
8
9  export async function CriarAlunos(request: Request, response: Response) {
10     const alunos = await alunoService.criar();
11     response.status(201).json(alunos);
12 }
```



Organização de código

- Camadas de abstração
 - Index
 - Router
 - Controller
 - **DTO** →
 - Service
 - Repositório
 - Entidade
- 
- Validação de dados
 - Tipagem de entrada e saída
 - Passagem de dados entre camadas



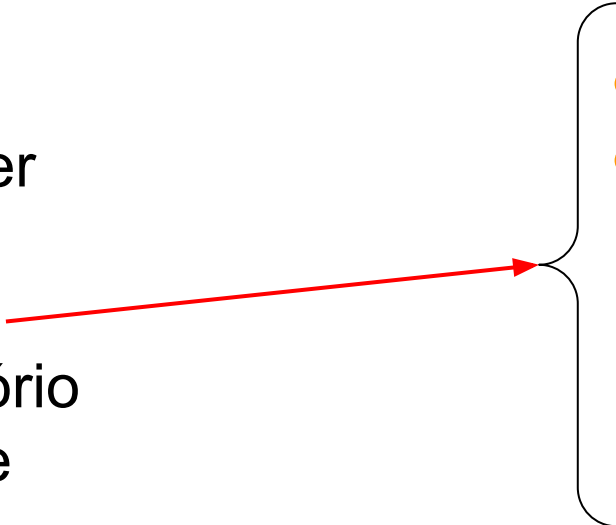
Organização de código

- Javascript
 - Não tem tipagem
 - DTO
 - Obj dos dados esperados

```
1  class AlunoDTO {
2      constructor({ nome, matricula, cursos = [] }) {
3          this.nome = nome;
4          this.matricula = matricula;
5          this.cursos = cursos;
6      }
7
8      static validar(dados) {
9          if (!dados.nome) {
10             throw new Error("O campo 'nome' é obrigatório.");
11          }
12          if (!dados.matricula) {
13             throw new Error("O campo 'matricula' é obrigatório.");
14          }
15      }
16  }
17
18  module.exports = AlunoDTO;
```



Organização de código

- Camadas de abstração
 - Index
 - Router
 - Controller
 - DTO
 - **Service**
 - Repositório
 - Entidade
- 
- Regras de negócios
 - Intermédio entre o controller e o repositório

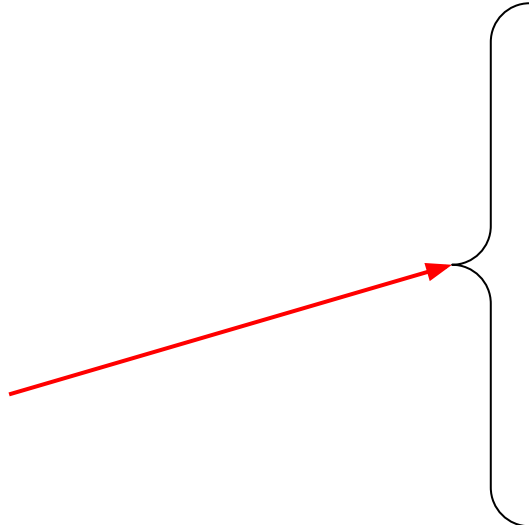


Organização de código

```
1  const db = require('../db/banco.js');
2  const Entidades = require('../db/entidades.enum.js');
3  const AlunoDTO = require('../dto/aluno.dto.js');
4  const util = require('../util/util.js');
5
6  class AlunoService {
7      static listar(pag, total) {
8          return util.paginacao(db.findAll(Entidades.aluno), pag, total);
9      }
10
11     static visualizar(id) {
12         return db.find(Entidades.aluno, id);
13     }
14 }
```




Organização de código

- Camadas de abstração
 - Index
 - Router
 - Controller
 - DTO
 - Service
 - **Repositório**
 - Entidade
- 
- Intermédio entre o service e o BD
 - Abstração do Acesso aos Dados
 - Consultas e Transações

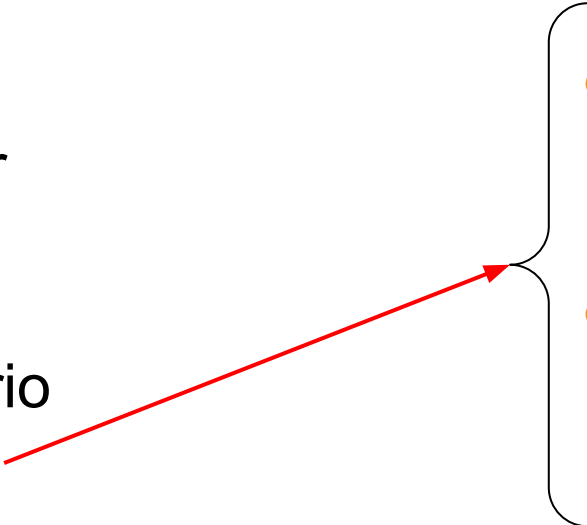


Organização de código

```
1  const db = require('../db/banco.js');
2  const Entidades = require('../db/entidades.enum.js');
3
4  class AlunoRepository {
5      static findAll() {
6          return db.findAll(Entidades.aluno);
7      }
8
9      static findById(id) {
10         return db.find(Entidades.aluno, id);
11     }
```



Organização de código

- Camadas de abstração
 - Index
 - Router
 - Controller
 - DTO
 - Service
 - Repositório
 - Entidade
- 
- Representação da entidade do banco de dados
 - Regras de dados



Middleware

- Interceptam e processam requisições e respostas no ciclo da API
 - Manipular, modificar ou realizar ações sobre os dados
 - Autenticação e Autorização
 - Mudar formato (JSON)
 - Logging
 - Validação



Middleware

- Parâmetros
 - Req
 - Res
 - Next()

Para um endpoint

```
1 rotas.get('/alunos', logMiddleware, controller.listar);
```

```
1 app.use(logMiddleware)
```

Para todo API

```
1 const logMiddleware = (req, res, next) => {  
2   const { method, url } = req;  
3  
4   console.log(`[${new Date().toISOString()}] ${method} - ${url}`);  
5   next();  
6 };  
7  
8 module.exports = logMiddleware;
```



Exercício

- Criar um simples api com um CRUD de alunos
 - Criar um array para simular o banco de dados
 - GET (listar e visualizar)
 - PUT, PATCH
 - DELETE
 - POST